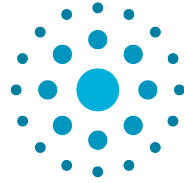


Université Côte d'Azur



UNIVERSITÉ
CÔTE D'AZUR

Multi-Agent Reinforcement Learning: a Simplified Padel Game

Giulia Raffaella Vitale

Student ID: 22506892

Subject: Advanced AI

Professor: Christopher Leturc

Contents

1	Introduction	2
2	Motivations and Development Choices	3
2.1	Motivations	3
2.2	Development choices	3
3	Environment and agents	5
3.1	Environment Setup	5
3.2	Markov Decision Process (MDP) Formulation	6
3.2.1	Formal Definition of the Multi-Agent MDP	7
3.2.2	Agent Types and Roles	8
3.2.3	Modelization	8
4	Training	11
4.1	Curriculum Learning Framework	11
4.2	Phase 1: Single-Agent Ball Interaction Learning	12
4.2.1	Player1step	12
4.2.2	Racket1Step	12
4.2.3	BallCurriculum1step	13
4.2.4	Phase 1 YAML Configuration	13
4.3	Phase 2: Full 1vs1 Competitive Learning	13
4.3.1	P1vs1	13
4.3.2	GM1vs1 (Game Manager)	14
4.3.3	BallController	14
4.3.4	Racket	14
4.3.5	Phase 2 YAML Configuration	15
4.4	Reward Architecture	15
5	Results	17
5.1	Observed Agent Behavior During Inference	17
5.1.1	Offensive Capabilities	17
5.1.2	Defensive and Reception Behavior	18
5.1.3	Rally Length and Game Dynamics	18
5.2	Analysis of the Observed Limitations	18
5.2.1	Curriculum Gap Between Phases	18
5.2.2	Self-Play Dynamics	19
5.3	Discussion and Future Improvements	19
6	Conclusions	20

1 Introduction

Multi-Agent Systems (MAS) and Multi-Agent Reinforcement Learning (MARL) provide a tractable framework for simulating environments where multiple autonomous entities interact, compete, and cooperate. These systems are increasingly significant for modeling complex, real-world scenarios that are otherwise difficult to analyze formally. This project leverages MARL to develop and analyze a system simulating a simplified 1v1 version of the sport of Padel, a domain characterized by its dynamic and physically rich interactions.

The decision to model a simplified version of the sport was a strategic necessity. The full 2v2 game of Padel involves high-dimensional state-action spaces, complex team dynamics, and highly non-linear physics, making it a formidable modeling challenge. By reducing the scenario to a 1v1 match, we isolate the core adversarial dynamics, creating a controlled environment for analysis. This simplification enables a focused study of competitive learning principles without the confounding variables of cooperative team play.

The primary objectives of this project are twofold: first, to explore the potential for emergent strategic behaviors to arise from agent interaction, free from the constraints of human cognitive bias; and second, to conduct experiments on agent learning within a challenging, physics-rich simulation. In doing so, the aim is to gain insights into how autonomous agents can master complex physical tasks and adapt their strategies in a competitive setting. This report details the underlying motivations, technical decisions, and formal modeling that have shaped this research.

2 Motivations and Development Choices

The foundational design choices in any simulation project are critical, as they directly define the scope and potential for discovering meaningful agent behaviors. The selection of the technical stack, the simplification of the ruleset, and the underlying strategic motivations collectively create the framework within which learning can occur.

2.1 Motivations

The project is guided by four fundamental research motivations that explore the intersection of AI, physics simulation, and competitive strategy:

- **Strategic Discovery:** by modeling the game without the preconceived notions or habits of human players, this Multi-Agent Reinforcement Learning (MARL) approach is designed to allow agents to discover optimal, and potentially non-obvious, strategies. The agents learn purely through trial and error, guided by the reward function, enabling them to identify the most effective court positioning and shot selections to maximize their probability of winning a point.
- **Adversarial Logic Sandbox:** the 1v1 match environment serves as an ideal "sandbox" for studying adversarial logic. In this competitive setting, each agent's success is directly dependent on its ability to anticipate and counter the actions of its opponent. This creates a dynamic feedback loop where agents must continuously adapt their tactics in real-time, providing a rich context for analyzing the development of intelligent, responsive behaviors.
- **Computational Efficiency:** simplifying the scenario to a 1v1 match allows for the effective isolation of key variables, such as the game's physics, court geometry, and agent-ball interactions. This reduction in complexity makes the training process more computationally tractable and enables a more focused and efficient analysis of the MARL algorithm's performance in learning the core mechanics of the sport.
- **Human-AI Synergy:** in the long term, this research has the potential to serve as a foundation for creating sophisticated training tools for human Padel athletes. The strategies and patterns discovered by the AI agents could be used to generate novel training drills, provide tactical insights, and simulate a wide variety of opponent playstyles, ultimately enhancing human performance.

2.2 Development choices

The development choices were centered around creating a high-fidelity simulation environment capable of accurately representing the physics of Padel while seamlessly integrating with advanced reinforcement learning frameworks.

- **Unity Engine:** Unity was selected as the primary simulation platform due to its robust, built-in rigid-body physics engine. This engine is essential for

simulating the non-linear trajectories and high-velocity collisions inherent to Padel ball dynamics, including bounces off the court and walls, gravity, and agent-environment interactions. Its powerful 3D rendering capabilities also allow for the creation of realistic environments for observation and analysis.

- **Unity ML-Agents Toolkit:** The ML-Agents toolkit is the essential framework that connects the Unity simulation environment with the Python-based training backend. It provides a highly optimized communication pipeline for states, actions, and rewards, which is critical for the high-throughput data collection required to train complex policies in a reasonable timeframe. It facilitates the implementation of advanced MARL algorithms such as Multi-Agent POsthumous Credit Assignment (MA-POCA). This algorithm is designed to solve the Posthumous Credit Assignment problem, where an agent that terminates early (e.g., by making a rally-ending mistake) must still learn from the subsequent success or failure.

These foundational choices in technology and research direction pave the way for a detailed examination of the environment and agent architecture.

3 Environment and agents

This section details the formal construction of the simulation environment and the agents designed to operate within it. To provide a rigorous basis for the reinforcement learning task, the entire system is modeled as a Markov Decision Process (MDP), which formally defines the relationship between the agents, their perceptions, their actions, and the outcomes they experience.

3.1 Environment Setup

The simulation takes place on a 3D Padel court designed to replicate the key features of a real-world playing area. The environment was constructed with the following components:

- A 3D model of the court serves as the primary playing surface. The environment is enclosed by 5 containing GameObjects, which form a bounding box to prevent the ball and agents from leaving the playable area.
- Four distinct trigger zones (`isTrigger=true`) are used to precisely manage game logic and reward assignment. These zones differentiate the in-play fields (red and blue) from the out-of-bounds areas, allowing the system to accurately determine the outcome of each shot.
- Specific spawn points are defined for each agent and the ball to reset the environment at the start of an episode or after a point is scored. These include positions like *redServingPosition* for the red agent and *ballRedServing* for the ball.
- The agents are represented by colored capsules (red and blue), while the ball and rackets are implemented as distinct 3D objects with their own physical properties.

Rule set defining and regulating agent behavior within the simulation environment:

- **Match Structure and Scoring**
 - **Episode Flow:** an episode begins with a serve and ends when a point is awarded or a fault occurs.
 - **Alternating Service:** the system alternates the serving team at the conclusion of every rally sequence.
 - **Reward Distribution:** a positive reward is given to the winner of the rally, and a penalty is given to the loser.
- **Ball-Ground Interaction**
 - **Valid Bounce:** the ball must hit the opponent’s side of the court first after being hit. A successful first bounce on the correct side grants a reward.

- **Double Bounce:** if the ball bounces twice on one side of the court, the player on that side loses the point.
 - **Out of Bounds:** touching the "outField" (the floor outside the court) is a point-ending event: if it touches the outField before bouncing in the opponent's court, the hitter loses the point; if it touches the outField after a valid bounce (and is not returned), the hitter wins the point.
- **Wall and Net Rules**
 - **The Net:** Hitting the net directly ends the rally and penalizes the last player who touched the ball.
 - **Walls:** Hitting the "courtNet" (likely the side fences/walls) has a specific logic: if it has already bounced once, it counts as a second bounce, the ball remains "receivable," and the opponent is permitted to hit the ball back to continue the rally; otherwise, it ends the rally if hit directly.
 - **Body Collision:** In padel, you cannot hit the ball with your body. The script enforces this with a penalty if the ball touches the body of the agent (not the racket).

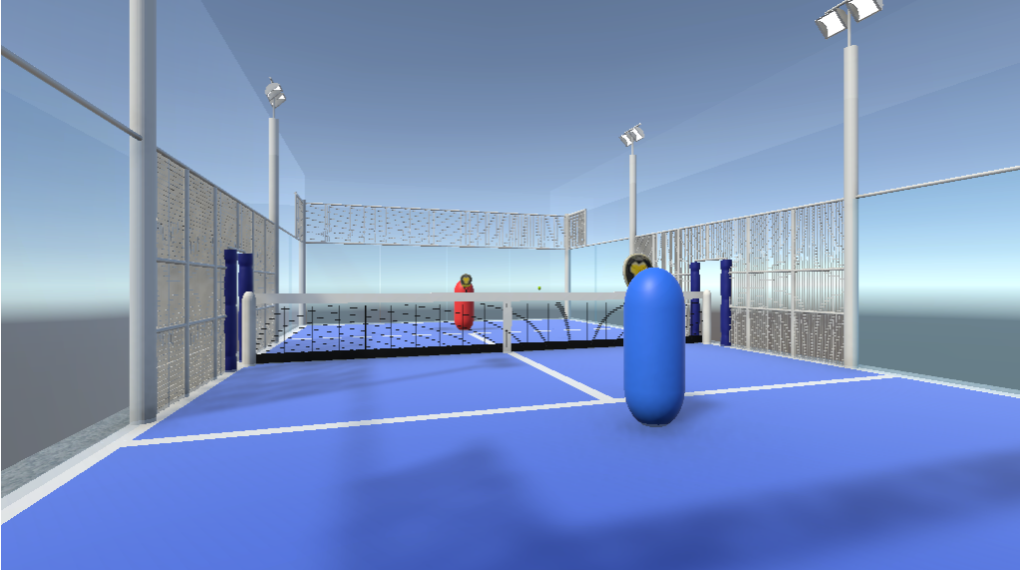


Figure 1: Environment.

3.2 Markov Decision Process (MDP) Formulation

The interaction between the agents and the environment is formally defined using the components of a Markov Decision Process. This framework provides the mathematical foundation for applying reinforcement learning algorithms.

- **State Space (S):** The state is the set of all possible observations an agent can perceive about the environment. Based on the CollectObservations method in the P1vs1.cs script, an agent’s state is represented by a vector containing the following key information:
 - The agent’s own local position and linear velocity.
 - The ball’s position relative to the agent and the ball’s linear velocity.
 - Contextual information, including a flag (isPlayerServing) indicating whether the agent is currently serving.
 - The opponent’s position relative to the agent and the opponent’s linear velocity.
- **Action Space (A):** The action space defines the set of all possible actions an agent can take. As defined in the OnActionReceived method, the agents operate using a vector of three continuous actions. However, the implemented logic utilizes three effective actions:
 - Movement along the X-axis (sideways), applied as a force.
 - Movement along the Z-axis (forward/backward), applied as a force.
 - A racket swing action, controlling the local rotation of the racket pivot.
- **Transition Function (P):** The transition function, which determines the next state of the environment given the current state and an action, is not explicitly programmed. Instead, the transition dynamics are implicitly governed by the Unity physics engine. This engine handles all outcomes of agent actions, including agent movement, ball trajectory calculations, gravitational effects, and collisions between objects.

3.2.1 Formal Definition of the Multi-Agent MDP

The system is modeled as a **Multi-Agent Markov Decision Process (MAMDP)**:

$$\mathcal{M} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \{\mathcal{R}_i\}_{i \in \mathcal{N}}, \rangle$$

where:

- $\mathcal{N} = \{R, B\}$ is the set of agents (Red player, Blue player),
- $\mathcal{S}$ is the global environment state space,
- $\mathcal{A}_i$ is the action space of agent i ,
- $\mathcal{P}$ is the state transition probability function,
- $\mathcal{R}_i$ is the reward function of agent i .

3.2.2 Agent Types and Roles

The multi-agent system previously described contains:

- **Learning agents**

- P1vs1 (Red player),
- P1vs1 (Blue player).

Each agent is:

- Autonomous,
- Goal-directed (maximize individual reward),
- Competitive,
- Reactive and learning-based.

- **Environment Agents (Non-learning)**

- Ball (BallController),
- Game Manager (GM1vs1),
- Court, net, grid, out-of-field zones.

3.2.3 Modelization

State Space $\mathcal{S}$

The global state $s \in \mathcal{S}$ is composed of:

$$s = (s_R, s_B, s_{ball}, s_{game})$$

- **Agent State**

For each agent $i \in \{R, B\}$:

$$s_i = \left[\underbrace{\mathbf{p}_i}_{\text{position}}, \underbrace{\mathbf{v}_i}_{\text{velocity}}, \underbrace{\mathbf{p}_{ball} - \mathbf{p}_i}_{\text{ball position}}, \underbrace{\mathbf{v}_{ball}}_{\text{ball velocity}}, \underbrace{\mathbb{I}_{serve}}_{\text{serving flag}}, \underbrace{\mathbf{p}_{opp} - \mathbf{p}_i}_{\text{opp position}}, \underbrace{\mathbf{v}_{opp}}_{\text{opp velocity}} \right]$$

where:

- $\mathbf{p}_i$: agent position,
- $\mathbf{v}_i$: agent velocity,
- $\mathbf{p}_{ball}, \mathbf{v}_{ball}$: ball position and velocity,
- $\mathbb{I}_{serve}$: serving indicator,
- $\mathbf{p}_{opp}, \mathbf{v}_{opp}$: opponent state.

- **Ball State**

$$s_{ball} = (\mathbf{p}_{ball}, \mathbf{v}_{ball}, b_c, b_r)$$

where b_c is the bounce count and b_r indicates receivability.

- **Game State**

$$s_{game} = (g_s, t, T_s)$$

with:

- $g_s \in \{\text{Serve, Rally}\}$,
- t elapsed rally time,
- T_s serving team.

Action Space $\mathcal{A}_i$

Each agent uses a continuous action space:

$$a_i = (a_x, a_z, a_{swing}) \in \mathbb{R}^3$$

where:

- a_x : movement along X-axis,
- a_z : movement along Z-axis,
- a_{swing} : racket swing activation.

Transition Function $\mathcal{P}$

$$\mathcal{P}(s' \mid s, a_R, a_B)$$

Transitions are:

- Deterministic given physics and actions,
- Affected by Unity's rigid-body dynamics,
- Event-driven (collisions, triggers).
- implicitly governed by game rules implemented in the game manager and ball manager.

Reward Functions $\mathcal{R}_i$

Each agent has an individual reward:

$$\mathcal{R}_i : \mathcal{S} \times \mathcal{A}_R \times \mathcal{A}_B \rightarrow \mathbb{R}$$

- **Positive Rewards**

- Successful racket hit,
- Good timing (ball height),
- Moving toward the ball,
- Valid bounce,
- Winning a rally.

- **Negative Rewards**

- Net or grid collision,
- Out-of-field hits,
- Invalid bounce,
- Inactivity when receiving,
- Losing a rally.

The game is **general-sum but competitive**, approximating a zero-sum structure at rally end.

Episode Termination

An episode ends when:

- a rally finishes (point scored),
- the ball touches the out-of-field area,
- the ball bounces twice on the same side,
- an invalid bounce occurs,
- the net or a wall is hit,
- maximum rally time is exceeded.

At termination:

- Both agents call `EndEpisode()`,
- The environment resets to a serve state.

This formal modeling of the agent and its environment provides a clear structure for applying the training methodology detailed in the following section.

4 Training

The agents were trained using a structured, two-phase curriculum learning approach. This methodology was necessary because attempting to train agents in a full 1v1 match from scratch often fails due to the sparse reward of winning a point, which is too difficult for an untrained agent to achieve. The curriculum was therefore designed as a form of reward shaping and skill decomposition, allowing agents to first build foundational skills in a dense-reward sub-task before introducing the complexities of direct adversarial interaction. The training was driven by the Multi-Agent POsthumous Credit Assignment (MA-POCA) algorithm.

MA-POCA is a centralized-training, decentralized-execution multi-agent reinforcement learning algorithm specifically designed to address the credit assignment problem in environments where multiple agents jointly influence a delayed and sparse outcome. Instead of attributing rewards only to the actions taken immediately before a terminal event, POCA retrospectively assigns credit to all agents and time steps that contributed to the final outcome of an episode. This is achieved by optimizing a centralized value function that conditions on the joint observations and actions of all agents, while still allowing each agent to learn an independent policy for execution.

The use of POCA is particularly well suited to the 1v1 padel scenario, where the success or failure of a rally depends on long sequences of coordinated and adversarial interactions, such as positioning, shot timing, and ball placement. In this setting, rewards such as winning or losing a point are only observed at the end of a rally, making it difficult for standard independent learning algorithms to correctly assign responsibility for the outcome. By propagating terminal rewards backward across the entire rally, MA-POCA enables more stable learning and encourages behaviors that are beneficial in the long term, rather than myopic reactions to immediate rewards. As a result, the algorithm facilitates the emergence of coherent strategies and robust competitive behavior in the full 1v1 match setting.

4.1 Curriculum Learning Framework

The curriculum is divided into two distinct phases, each with a specific learning objective.

- **Phase 1, Foundational Skill Acquisition:** In this initial phase, detailed in the `Player1step.cs` and `BallCurriculum1step.cs` scripts, a single agent is trained in a solo environment. The primary goal is for the agent to master the fundamental mechanics of the game: moving to intercept the ball, timing its swing correctly, and aiming its return shot into the opponent’s court. This phase focuses on the dense-reward sub-task of making contact and aiming before tackling the sparse-reward primary task of winning a rally.
- **Phase 2, Adversarial Training:** Following the acquisition of basic skills, the training progresses to a full 1v1 competitive scenario, managed by the `P1vs1.cs`

and GM1vs1.cs scripts. Two agents are placed in the environment and tasked with competing against each other. The objective shifts from simply hitting the ball to sustaining a rally, winning points, and dynamically reacting to the strategic actions of an intelligent opponent.

4.2 Phase 1: Single-Agent Ball Interaction Learning

The objective of Phase 1 is to teach an agent how to move, approach the ball, and perform a valid racket hit. The environment is simplified to remove the opponent and complex game rules, allowing stable learning of core behaviors.

4.2.1 Player1Step

`Player1Step` extends the `Agent` class and represents the learning agent during Phase 1.

Role This script controls the agent’s movement, observations, action execution, and reward assignment. It is responsible for learning how to position itself relative to the ball and perform a correct hit toward the opponent’s field.

Observations The agent observes:

- Its own local position and linear velocity.
- The relative position and velocity of the ball.
- A set of neutral (zero-valued) observations to preserve compatibility with the larger observation space used in Phase 2.

Actions The agent uses continuous actions to:

- Move along the X and Z axes.
- Perform a swing action that rotates the racket.

Rewards and Penalties

- Positive reward for hitting the ball with the racket.
- Additional reward for good timing (ball height within a valid range).
- Shaped reward for reducing the distance to the ball.
- Penalty for collisions with the net, grid, or court boundaries.

The episode resets after terminal events such as out-of-field hits or valid scoring outcomes.

4.2.2 Racket1Step

Role `Racket1Step` is a collision handler attached to the racket collider.

Function When the racket collides with the ball, this script notifies the agent by invoking `OnRacketHit`. This separation simplifies collision logic and keeps the agent script focused on learning-related behavior.

4.2.3 BallCurriculum1step

Role This script manages ball-related terminal conditions and assigns sparse, outcome-based rewards.

Functionality

- Detects when the ball lands in the correct opponent field, granting a positive terminal reward.
- Penalizes out-of-field events, net collisions, or ground contacts on the agent’s side.
- Ends the episode after any terminal condition.

4.2.4 Phase 1 YAML Configuration

The Phase 1 training configuration uses the POCA (Policy Optimization with Centralized Critics) trainer, despite involving a single learning agent, to maintain consistency with Phase 2.

Key Characteristics

- Moderate buffer and batch sizes to stabilize early learning.
- A shared network architecture (256 hidden units, 2 layers) reused in Phase 2.
- No self-play enabled.
- Training focuses exclusively on extrinsic rewards.

4.3 Phase 2: Full 1vs1 Competitive Learning

Phase 2 introduces a second agent, full game rules, and adversarial self-play. The goal is to learn competitive padel strategies while preserving the motor skills acquired in Phase 1.

4.3.1 P1vs1

`P1vs1` represents a player agent in the full 1vs1 environment.

Role Each instance controls one player (red or blue team), handling movement, observations, swing actions, and local reward shaping.

Observations The observation space includes:

- Self position and velocity.
- Relative ball position and velocity.
- A serving-state flag.
- Opponent position and velocity.

Rewards

- Reward for valid racket hits and good timing.
- Shaped reward for approaching the ball when receiving.
- Penalty for inactivity, invalid positioning, or illegal collisions.

4.3.2 GM1vs1 (Game Manager)

Role GM1vs1 orchestrates the entire match logic.

Responsibilities

- Manages serving alternation and rally states.
- Assigns win/loss rewards.
- Handles episode termination and environment resets.
- Acts as a centralized authority for reward assignment.

This centralized structure simplifies multi-agent credit assignment.

4.3.3 BallController

Role The BallController tracks ball dynamics and enforces padel-specific rules.

Functionality

- Detects valid and invalid bounces.
- Tracks expected receivers and rally state.
- Penalizes body collisions and illegal contacts.
- Rewards successful receptions and extended rallies.

The script provides critical context used by agents to condition their behavior.

4.3.4 Racket

Role This script links physical ball collisions to agent logic during Phase 2.

Function On ball contact, it:

- Transitions the game state from serve to rally.
- Triggers the agent’s racket hit logic.

4.3.5 Phase 2 YAML Configuration

Phase 2 extends the Phase 1 configuration with self-play to enable competitive learning.

Key Features

- Larger buffer size to accommodate multi-agent dynamics.
- Same neural network architecture as Phase 1, enabling weight reuse.
- Self-play with Elo rating, opponent swapping, and historical model sampling.

Self-play allows agents to co-evolve strategies and prevents overfitting to a single opponent.

4.4 Reward Architecture

A carefully designed reward and penalty structure guides the agents’ learning process. The functions are tailored to each phase of the curriculum, with values sourced from constants defined across the project scripts.

For each agent $i \in \{R, B\}$, the instantaneous reward at time step t is decomposed into the following components:

$$R_i = R_i^{\text{movement}} + R_i^{\text{racket}} + R_i^{\text{timing}} + R_i^{\text{bounce}} + R_i^{\text{outcome}} + R_i^{\text{penalty}}$$

where the components are:

- **Movement-based reward (approaching the ball).** When agent i is the expected receiver, it is rewarded for reducing its distance to the ball:

$$R_i^{\text{move}}(t) = \alpha_m (d_{t-1}^i - d_t^i),$$

where:

- $d_t^i = \|\mathbf{p}_i(t) - \mathbf{p}_b(t)\|$ is the Euclidean distance between agent i and the ball at time t ,
- $\mathbf{p}_i(t) \in \mathbb{R}^3$ is the position of agent i ,
- $\mathbf{p}_b(t) \in \mathbb{R}^3$ is the position of the ball,
- $\alpha_m > 0$ is a scaling coefficient for movement shaping.
- **Racket–ball contact reward.** When agent i successfully hits the ball with the racket:

$$R_i^{\text{hit}} = \alpha_h,$$

where $\alpha_h > 0$ is a constant reward encouraging active ball interception.

- **Timing reward for valid hit height.** If the ball height at the instant of racket contact satisfies:

$$h_{\min} < y_b < h_{\max},$$

the agent receives:

$$R_i^{\text{timing}} = \alpha_t,$$

where:

- y_b is the vertical coordinate of the ball at contact,
 - $h_{\min}$ and $h_{\max}$ define the valid hitting window,
 - $\alpha_t > 0$ rewards well-timed strokes.
- **Bounce-related reward and penalty.** Ball bounces are evaluated according to padel rules:

$$R_i^{\text{bounce}} = \begin{cases} +\alpha_b, & \text{if the ball bounces once in the opponent court,} \\ -\beta_b, & \text{if the bounce is invalid (wrong side or illegal).} \end{cases}$$

Here:

- $\alpha_b > 0$ encourages legal, well-placed shots,
 - $\beta_b > 0$ penalizes rule violations.
- **Rally outcome reward.** At the end of a rally, the agent receives:

$$R_i^{\text{outcome}} = \begin{cases} +\alpha_w, & \text{if agent } i \text{ wins the point,} \\ -\beta_l, & \text{if agent } i \text{ loses the point.} \end{cases}$$

where:

- $\alpha_w > 0$ is the winning reward,
 - $\beta_l > 0$ is the losing penalty.
- **Penalties for unsafe or undesirable behavior.** Additional penalties are applied as:

$$R_i^{\text{penalty}} = -\beta_n \mathbb{I}_{\text{net}} - \beta_o \mathbb{I}_{\text{out}} - \beta_c \mathbb{I}_{\text{body}} - \beta_{\text{idle}} \mathbb{I}_{\|\mathbf{v}_i\| < \varepsilon},$$

where:

- $\mathbb{I}_{\text{net}}$ indicates collision with the net or court grid,
- $\mathbb{I}_{\text{out}}$ indicates illegal out-of-field contact,
- $\mathbb{I}_{\text{body}}$ indicates ball-body collision,
- $\mathbb{I}_{\|\mathbf{v}_i\| < \varepsilon}$ indicates agent inactivity,
- $\mathbf{v}_i$ is the agent velocity,
- $\varepsilon > 0$ is a small inactivity threshold,
- $\beta_n, \beta_o, \beta_c, \beta_{\text{idle}} > 0$ are penalty coefficients.

5 Results

The curriculum learning framework and reward architecture are specifically designed to produce a set of observable, competent behaviors. Based on the training methodology, we anticipate the agents will demonstrate proficiency in several key areas of the game.

The expected outcomes are as follows:

- **Successful Rally Maintenance:** Agents are expected to learn the fundamental skill of consistently returning the ball into the opponent’s court, thereby sustaining rallies for multiple exchanges.
- **Emergent Defensive Positioning:** After executing a shot, agents should demonstrate intelligent movement patterns. We expect to see them learn to return to a central or otherwise advantageous position on the court to prepare for the opponent’s subsequent return.
- **Adaptive Shot Response:** The agents are expected to develop the capacity to adapt their position and response based on the opponent’s actions, successfully handling varying ball trajectories, speeds, and placements.

5.1 Observed Agent Behavior During Inference

To evaluate the learned policies, an inference-only scene was constructed using the trained ONNX models. The scene consists of a full padel court prefab and two agents, each assigned a trained policy and configured with **Behavior Type = Inference**. No learning or reward signals were active during this phase, ensuring that the evaluation reflects the final learned behavior.

After approximately 100 hours of training across both curriculum phases, the agents demonstrated several consistent and observable behaviors.

5.1.1 Offensive Capabilities

One of the most reliable behaviors exhibited by the agents was their ability to strike the ball and send it into the opponent’s court. In many episodes, the agents successfully:

- Approached the ball when it was within reachable distance.
- Executed a swing action with correct timing relative to ball height.
- Generated trajectories that cleared the net and landed within the adversary’s field.

These results indicate that the motor control, swing timing, and basic directional understanding acquired during Phase 1 were effectively transferred to the full 1vs1 environment. The agents appear to have learned a stable and repeatable action pattern for ball striking, suggesting that the reward shaping related to racket hits and valid ball placement was effective.

5.1.2 Defensive and Reception Behavior

In contrast, the agents exhibited limited competence in receiving opponent shots. During inference, it was frequently observed that:

- Agents failed to move toward the incoming ball after the opponent’s hit.
- The ball was often allowed to bounce multiple times without an attempt to intercept.
- Successful receptions occurred only sporadically and appeared inconsistent rather than intentional.

This behavior suggests that, although the agents can execute offensive actions, they did not develop a robust policy for defensive positioning and reception. In many cases, agents remained stationary or moved in suboptimal directions while the ball traveled toward their side of the court.

5.1.3 Rally Length and Game Dynamics

As a direct consequence of the limited reception capability, rallies during inference were generally short. Typical exchanges consisted of:

- A successful serve or return by one agent.
- A valid bounce in the opponent’s field.
- Failure of the receiving agent to intercept the ball.

Extended rallies involving multiple back-and-forth exchanges were rare. This outcome deviates from the expected rally maintenance behavior described earlier and highlights an imbalance between offensive and defensive skill acquisition.

5.2 Analysis of the Observed Limitations

The observed performance gap between ball striking and ball reception can be attributed to several factors related to the curriculum design and reward structure.

5.2.1 Curriculum Gap Between Phases

While Phase 1 successfully taught the agent how to approach and hit a stationary or predictable ball, it did not explicitly train the agent to receive a dynamically incoming shot. Phase 2 introduced reception as part of a fully competitive 1vs1 scenario, significantly increasing task complexity by simultaneously requiring:

- Opponent modeling.
- Ball trajectory prediction.
- Strategic movement under adversarial conditions.

A missing intermediate curriculum phase focused solely on receiving the ball without adversarial pressure may have limited the agent’s ability to generalize this skill.

5.2.2 Self-Play Dynamics

Although self-play is essential for competitive learning, it can also amplify early-stage policy weaknesses. If both agents converge toward similar offensive-focused strategies, reception behavior may not be sufficiently reinforced, leading to a stable yet suboptimal equilibrium in which rallies end quickly.

5.3 Discussion and Future Improvements

Despite the extended training time, the results indicate that the curriculum learning strategy, while effective for offensive skill acquisition, was insufficient to produce balanced and robust 1vs1 gameplay.

A key improvement would be the introduction of an additional curriculum phase between Phase 1 and Phase 2, specifically designed to:

- Train the agent to receive incoming balls with varied trajectories.
- Emphasize defensive positioning and movement anticipation.
- Remove competitive pressure while preserving dynamic ball motion.

Due to time constraints, this intermediate phase was not implemented. Nevertheless, the results provide valuable insight into the challenges of multi-agent curriculum design and highlight the importance of explicitly training defensive skills in adversarial environments.

6 Conclusions

This work explored the use of curriculum learning and multi-agent reinforcement learning to train autonomous agents in a simplified 1vs1 padel environment using Unity ML-Agents. The learning process was structured into two curriculum phases, progressing from isolated ball interaction to full adversarial gameplay with self-play.

The results show that the proposed approach successfully enabled the agents to acquire fundamental offensive skills. During inference, the agents consistently demonstrated the ability to approach the ball, execute well-timed swings, and send valid shots into the opponent’s court, indicating effective transfer of low-level motor and interaction skills learned during the early curriculum phase.

However, the agents exhibited limited capability in receiving opponent shots. Defensive behaviors such as anticipation, repositioning, and interception were inconsistent, leading to short rallies dominated by single-shot exchanges. This outcome suggests that the transition from isolated skill learning to full 1vs1 competition introduced a level of complexity that hindered the emergence of robust reception behavior.

Overall, this study highlights both the potential and the challenges of applying curriculum learning to physically grounded multi-agent sports simulations. Future work should focus on refining curriculum design and reward structures to support more balanced gameplay, possibly adding an intermediate curriculum phase focused exclusively on reception and defensive movement.

A particularly promising extension of this project is the expansion to a full 2v2 padel scenario. Such an environment would require agents to learn not only adversarial strategies, but also effective cooperation with a teammate. Agents would need to coordinate positioning, shot selection, and defensive coverage while competing against an opposing team. Modeling this interplay between intra-team cooperation and inter-team competition represents a compelling and challenging direction for future research.